

4:27



LAB 1

Build First Android App

Enter first text

hi

Enter second text

hello

Check Match

Incorrect



KOTLIN BASICS.

AIM:

To understand the basic syntax and features of the Kotlin programming language.

ALGORITHM:

- * Start the android application.
- * Create a new android project in Android Studio using Kotlin and jetpack compose.
- * Import the required packages and libraries
- * Apply the application theme
- * Run the application in Android Emulator or physical device.
- * Verify the output
- * Stop the program.

RESULT:

| The basic Kotlin programs were implemented successfully and the fundamental syntax and operations were made clear and understood.

4:23

EXPERIMENT 2



LAYOUT



Line

Circle

Show All

Clear All

AIM:

To study and implement functions in Kotlin for performing specific task.

ALGORITHM:

- * Start the application
- * Create a new android project in Android studio using Kotlin and jetpack compose.
- * Import the required packages and libraries.
- * Apply the application theme.
- * Implement the required functionality using state variables and event handlers.
- * Run the application in android emulator.
- * Verify output.

RESULT:

Functions were created and executed successfully demonstrating modular programming in Kotlin.

LAB 3

App Navigation

Go to Second Page

Click the button above to navigate to another page

Second Page

Welcome to the second page!

Go Back to Main

Click the button above to return to the main page

AIM:

To understand object oriented programming concepts using classes and objects in Kotlin.

ALGORITHM:

- * Start the android application
- * Create a new Android project in Android Studio using Kotlin and Jetpack compose.
- * Import required packages and libraries.
- * Apply application theme.
- * Run the application in an Android Emulator or physical device.
- * Verify the output
- * Stop the program.

RESULT:

Classes and objects were created successfully and the concept of object oriented programming was understood.

← Typography

Hierarchy Matters

Display Large

Size: 57.0.sp • Weight: FontWeight(weight=400)

Headline Medium

Size: 28.0.sp • Weight: FontWeight(weight=600)

Title Large

Size: 22.0.sp • Weight: FontWeight(weight=500)

Body Large (Default)

Size: 16.0.sp • Weight: FontWeight(weight=400)

Label Small

Size: 11.0.sp • Weight: FontWeight(weight=500)

Font Weights & Styles

Normal Medium Bold *Italic*

Educational Tip: Use contrast in font size and weight to guide the user's eye to the most important content first.

BUILD FIRST ANDROID

APP.

AIM:

To create and run a simple android application using Android emulator & Android studio and Kotlin.

ALGORITHM:

- * Start the android application.
- * Create a new project in Android studio using Kotlin and Jetpack compose.
- * Import the required libraries and package.
- * Apply the application theme.
- * Run the application in android emulator or physical device.
- * Verify the output.
- * Stop the program.

RESULT:

A basic android application was created and executed successfully in emulator device.

4:29



LAB 5

App Architecture (UI Layer)

Button 1

Button 2

Card Example

This card shows padding
and margins

Rounded Box
with padding



LAYOUTS

AIM:

To design and implement layouts in an android application using jetpack compose.

ALGORITHM:

- * Start the android application
- * Create a new android project in android studio using Kotlin and Jetpack compose.
- * Import the required packages and libraries.
- * Apply the application theme.
- * Implement the required functionality using state variables and event handlers.
- * Run the application in an android emulator
- * Verify output
- * Stop program.

RESULT:

Layouts were successfully designed and UI components were arranged properly in the application.

4:30

LAB 6

App Architecture - Persistence

This app demonstrates different persistence methods in Android architecture.

Architecture Components

- **SharedPreferences**
Simple key-value pairs for primitive data
- **Internal Storage**
Files stored in app's private directory
- **Room Database**
Structured data storage with SQLite

SharedPreferences

Simple key-value storage for primitive data

Enter your name

Meenakshi

Save Name

Clear

Saved Name: Meenakshi

Internal Storage

Files stored in app's private directory

Internal Storage

Files stored in app's private directory

Enter a note

Save

Load

Delete

Current: Helloooo

Room Database

Structured data storage with SQLite backend

Task title

Task description

Add Task

MAD Lab

Task updation

Build Persistence Demo

Create an app showing different persistence methods

AIM:

To implement app architecture between different screens in an android application.

ALGORITHM:

- * Start the program
- * Create a new Android project in android studio using Kotlin and Jetpack Compose.
- * Add required dependencies for Navigation Compose in the build.gradle file.
- * Create multiple screens such as HomeScreen, Detail Screen etc.
- * Initialize NavController using rememberNavController()
- * Setup NavHost and connect it with the nav controller.
- * Add composable destinations inside NavHost and map them to destinations.
- * Implement navigation actions.
- * Handle back stack and state persistence to maintain navigation state across recompositions.
- * Run the application on an emulator or physical device.
- * Verify navigation flow between multiple screens.
- * Stop the program.

RESULT:

App architecture persistence between multiple screens was implemented successfully.

LAB 7

Advanced RecyclerView Use Cases



Multiple View Types

Combining Headers and Content items in one list.



DiffUtil & ListAdapter

Optimizing list updates with efficient background diffing.

INTERACTIONS



ItemTouchHelper

Implementing Swipe-to-Delete and Drag-and-Drop functionality.



Click Listeners

Handling complex item interactions and navigation.

PERFORMANCE



View Recycling

Understanding the ViewHolder pattern and view reusability.



Payload Updates

Updating only specific parts of an item without full Redraw.

LAB 7

Advanced RecyclerView Use Cases

GETTING STARTED



Basic RecyclerView

Setting up a simple list with a single view type.



View Binding

Using ViewBinding for cleaner view access in ViewHolders.

ADVANCED ARCHITECTURES



Multiple View Types

Combining Headers and Content items in one list.



DiffUtil & ListAdapter

Optimizing list updates with efficient background diffing.

INTERACTIONS



ItemTouchHelper

Implementing Swipe-to-Delete and Drag-and-Drop functionality.

Click Listeners

AIM:

To study the advanced recycler view use cases and implement the same.

ALGORITHM:

- * Start the android application and android studio to create a new Kotlin project.
- * Add the recyclerview dependency and design a layout containing a recycler view.
- * Create a data model and an adapter with view Holder to display the list items.
- * Add item interactions such as click listeners and swipe actions using itemtouchhelper.
- * Verify output by running application.

RESULT:

The advanced recyclerview usecases were observed and understood successfully.

LAB 8

Connect to the internet



Connected

Understanding Connectivity

This app uses `ConnectivityManager` and `NetworkCallback` to listen for real-time network changes. When the device gains or loses internet access, the UI updates automatically.

AIM:

To connect the android application to the internet and retrieve data from online resources.

ALGORITHM:

- * To create a new project start Android Emulator and Android application using Kotlin.
- * Add the required internet and network permissions in the file.
- * Use `ConnectivityManager` to monitor the network connection status.
- * Import `NetworkCallback` to detect when the device connects or disconnects from internet.
- * Update the UI to display the connection status.
- * Run the application and verify the status change.

RESULT:

The application connected successfully to internet and receive required data.

4:33



LAB 9

Repository Pattern and Work Manager

How it works:

1. Repository manages data from Room DB.
2. Work Manager runs SyncWorker in background.
3. UI updates automatically via Flows.

New Goal Sync

This goal was added in the background by Work Manager.

Synced at: 15:56:14, 09 Apr 2026

+ Sync Now

REPOSITORY PATTERN AND WORKMANAGER.

AIM:

To implement the repository pattern and create a new project with workmanager for background tasks in Android.

ALGORITHM:

- * Start the android application and create a new project in Android studio using Kotlin.
- * Create a room database and implement the repository pattern to manage data operations.
- * Create a workclass using workmanager to perform background tasks such as syncing data.
- * Connect the repository with workmanager to store and update data in database.
- * Run the application and verify.

RESULT:

Repository pattern and workmanager were implemented successfully for data management and background processing.

LAB 10

App UI Design

Learn & Understand UI Principles



Color Theory

Harmony, Contrast & Palette



Typography

Fonts, Scales & Readability



Layout & Spacing

Grids, Padding & Alignment



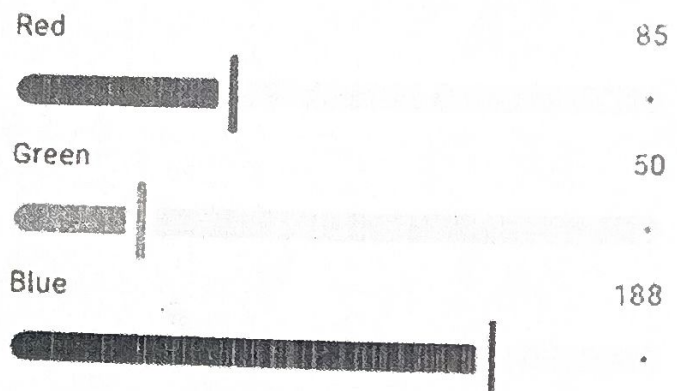
Components

Interactive UI Elements

← Color Theory

#5632BD

Interactive RGB Mixer



Principles of Color:

- Primary Colors: Red, Green, Blue (Digital)
- Harmony: Complementary colors create high contrast.
- Accessibility: Ensure text contrast ratios are met (WCAG).

AIM:

To design an attractive and user friendly interface for an android application.

ALGORITHM:

- * Start the android application and create a new project in Android studio using Kotlin.
- * Design the main UI layout using components such as button, text.
- * Implement UI design principles
- * Add interactive components
- * Apply proper alignment
- * Run the application.

RESULT:

The application UI was designed successfully with proper layout and styling.